



Minería de Datos

IV. Clustering analysis

Tarea 9

Algoritmo DBSCAN: clasificador de vinos

Utilice el mismo Notebook de Jupyter de la tarea K-means.

1. Ejecute todas las celdas para tener cargados los datos
En la barra de menús Run->Run all cells
2. Cree una tubería auxiliar (pipe2) y configure solo la etapa de preprocesamiento para que se aplique la normalización de MinMaxScaler y un análisis de componentes principales (PCA) para **seis** componentes.
preprocessor = Pipeline([
 ("scaler", MinMaxScaler()),
 ("pca", PCA(n_components=6, random_state=42)),])
pipe2 = Pipeline([("preprocessor", preprocessor)])
pipe2.fit(wine['data'])

3. Haga un análisis para determinar los hiper parámetros óptimos del modelo DBSCAN (eps y min_samples) pruebe con los siguientes rangos:
eps_values = np.arange(0.28,0.38,0.005)
min_samples = np.arange(3,18)
X_numerics=pipe2["preprocessor"].transform(wine['data'])

Al mostrar la matriz de calor del desempeño de **silueta** podrá notar que el máximo global es **0.24** con (eps=**0.375** y min_samples=**7**) sin embargo, observe que el número de clusters generados para esa combinación es solo 3 (incluido el cluster outlier). Como en este ejemplo tenemos 3 categorías de vinos 0-2 necesitamos elegir una combinación que genere 4 clusters por lo que podemos elegir (eps=**0.34** y min_samples=**9**) que tiene un desempeño de **0.23**

4. Cree una tubería con la misma configuración de preprocesamiento que la anterior y configure el algoritmo DBSCAN con los parámetros óptimos.
clusterer2 = Pipeline([
 "dbscan",
 DBSCAN(eps=0.34, min_samples=9),])
pipe2 = Pipeline([("preprocessor", preprocessor), ("clusterer", clusterer2)])
5. Entrene el modelo con los datos del dataset (wine['data'])
6. Consulte el resultado del agrupamiento de DBSCAN



```
predicted_labels = pipe2["clusterer"]["dbscan"].labels_  
predicted_labels
```

7. Realice las gráficas del agrupamiento real (wine['target']) y del agrupamiento con DBSCAN para analizar qué tan acertada fue la agrupación.

- y_predict = predicted_labels
- Data es wine['data']
- True_labels es wine['target']
- label_encoder.classes_ es labels=(1,2,3)

Por lo que las coordenadas x,y son:

```
x = pipe2["preprocessor"].transform(wine['data'])[wine['target'] == i, 0],  
y = pipe2["preprocessor"].transform(wine['data'])[wine['target'] == i, 1],
```

*Nota: los outliers son los puntos morados.

8. Genere la matriz de confusión para saber que tanto se equivocó el algoritmo.

Como podrá darse cuenta para este problema resultó mejor haber aplicado el agrupamiento K-means, pues DBSCAN aunque se equivocó menos identificó varios puntos como outliers al estar algo dispersos. La importancia de conocer varias técnicas de agrupamiento es poder aplicar la adecuada en determinada situación, siendo DBSCAN muy útil para detectar formas y outliers y K-means de propósito general.

9. Ahora repetiremos todo el procedimiento desde el paso 2 para analizar el desempeño de DBSCAN con la métrica Manhattan (cityblock).

En todas las líneas donde crea el algoritmo DBSCAN agregar el parámetro metric:

```
DBS_clustering=DBSCAN(eps=p[0], min_samples=p[1], metric="cityblock").fit(X_numerics)
```

Usar:

- eps_values = np.arange(0.5,0.75,0.01)
- min_samples = np.arange(3,13)

*Nota: A diferencia de *K-means* donde basta con indicar la métrica para entrenar el algoritmo con DBSCAN debe indicar los hiper parámetros que varían mucho de una métrica a otra por lo que solo analizaremos Manhattan.

Al mostrar la matriz de desempeño con **silueta** podrá notar que el máximo global es **0.26** con **eps>=0.71** pero la cantidad de clusters que genera ese valor de eps es solo 3 por lo que descartamos esa configuración. Revisando más hacia la izquierda en la matriz encontramos un valor de **0.22** con **eps=0.6** y **min_samples=4**; configure el modelo DBSCAN con estos parámetros y la métrica cityblock.